MINOR PROJECT-02

NAME:S. Gayathri Chandana

SUBMISSION DATE:04-09-26

COURSE: VLSI

PROJECT NAME: ALU WITH TESTBENCH

ALU WITH TESTBENCH

- **Introduction**

An **Arithmetic Logic Unit (ALU)** is an important part of a digital system. It is mainly used to perform arithmetic and logical operations on binary data. 4-bit ALU using Verilog HDL is designed in this project.

- **Operations Performed by the ALU:**

Our ALU performs 8 different operations. The operation is selected using a 3-bit ALU_Sel input.

ALU_Sel	Operation
000	Addition
001	Subtraction
010	AND
011	OR
100	XOR
101	NOT
110	Shift Left
111	Shift Right

- **Working f the ALU**

The ALU takes two 4-bit inputs, **A and B**, along with a 3-bit selection signal called ALU_Sel. The selection signal tells the ALU which operation to perform. We used a **case statement in Verilog** to select the required operation. For example, when ALU_Sel is 000, the ALU adds A and B. When it is 010, the ALU performs an AND operation. Similarly, the other selection values are used for OR, XOR, NOT, and shift operations. Whenever the input values or selection signal change, the output of the ALU changes according to the selected operation.

- **Simulation Results**

The waveform shows the changes in **A, B, ALU_Sel, and ALU_Out** for each operation.

The outputs obtained in simulation were:

Addition - 1000

Subtraction - 0010

AND -1000

OR -1110

XOR - 0110

NOT - 0101

Shift Left - 0110

Shift Right -0110

The waveform shows that the ALU produces the expected output for all the tested operations.

```
Ln#
 1  module ALU(
 2      input  [3:0] A,
 3      input  [3:0] B,
 4      input  [2:0] ALU_Sel,
 5      output reg [3:0] ALU_Out
 6  );
 7
 8  always @(*) begin
 9      case (ALU_Sel)
10
11          3'b000: ALU_Out = A + B;  // Addition
12          3'b001: ALU_Out = A - B;  // Subtraction
13          3'b010: ALU_Out = A & B;  // AND
14          3'b011: ALU_Out = A | B;  // OR
15          3'b100: ALU_Out = A ^ B;  // XOR
16          3'b101: ALU_Out = ~A;     // NOT A
17          3'b110: ALU_Out = A << 1; // Shift Left
18          3'b111: ALU_Out = A >> 1; // Shift Right
19
20          default: ALU_Out = 4'b0000;
21
22      endcase
23  end
24
25  endmodule
```

D:/alu_tb.v (/ALU_tb) - Default

```
Ln#
 1  `timescale 1ns/1ps
 2
 3  module ALU_tb;
 4
 5      reg [3:0] A;
 6      reg [3:0] B;
 7      reg [2:0] ALU_Sel;
 8      wire [3:0] ALU_Out;
 9
10      ALU uut (
11          .A(A),
12          .B(B),
13          .ALU_Sel(ALU_Sel),
14          .ALU_Out(ALU_Out)
15      );
16
17      initial begin
18
19          // Addition
20          A = 4'b0101;
21          B = 4'b0011;
22          ALU_Sel = 3'b000;
23          #10;
24
25          // Subtraction
26          A = 4'b0101;
27          B = 4'b0011;
```

```
Ln#
24
25          // Subtraction
26          A = 4'b0101;
27          B = 4'b0011;
28          ALU_Sel = 3'b001;
29          #10;
30
31          // AND
32          A = 4'b1100;
33          B = 4'b1010;
34          ALU_Sel = 3'b010;
35          #10;
36
37          // OR
38          A = 4'b1100;
39          B = 4'b1010;
40          ALU_Sel = 3'b011;
41          #10;
42
43          // XOR
44          A = 4'b1100;
45          B = 4'b1010;
46          ALU_Sel = 3'b100;
47          #10;
48
49          // NOT
50          A = 4'b1010;
```

```
Ln#
46          B = 4'b1010;
47          ALU_Sel = 3'b100;
            #10;
49
50          // NOT
51          A = 4'b1010;
            B = 4'b0000;
53          ALU_Sel = 3'b101;
            #10;
55
56          // Shift Left
57          A = 4'b0011;
            B = 4'b0000;
59          ALU_Sel = 3'b110;
60          #10;

62          // Shift Right
            A = 4'b1100;
64          B = 4'b0000;
65          ALU_Sel = 3'b111;
66          #10;

68          $stop;
69      end
70
71  endmodule
```

- The waveform shows that the ALU produces the expected output for all the tested operations.

